

Outline

- General organization of the course
- Object oriented programming
- Basic types and their operations

Methods

- UML class diagrams

Methods

Methods realize the **dynamic behavior** of objects and define functions

```
int average(int v1, int v2) {  
    return (v1 + v2) / 2;  
}
```

```
int fahrenheit(int celsius) {  
    return (celsius * 9) / 5 + 32;  
}
```

```
double voltage(double resistance, double current) {  
    return resistance * current;  
}
```

Method signature

- The interface of a mathematical function consists of the name of the function, an ordered list of (formal) parameters and their types, and the return type
- A method signature in Java consists of
 - Name of the method
 - Ordered list of (formal) parameters and their types (this determines the order and number)
 - Note: in Java, the return type is not part of the method signature
- Examples

```
int fahrenheit(int celsius) {  
    return (celsius * 9) / 5 + 32;  
}
```

Signature

Not part of
the signature

```
double fahrenheit(double celsius) {  
    return (celsius * 9) / 5 + 32;  
}
```


Method call

- **Example:** Ohm's law

Formal parameters

```
double v(double r, double i) {  
    return r * i;  
}
```

v: voltage
i: current
r: resistance

- **Call:**

```
double r1 = 3.0;  
double v1 = v(r1, 2.0);
```

Actual parameters

- **Syntax:** `methodname(actualparameter)`
 - **Result:** An element with the return type of the method
- What is the value of v1?

Example: cylinder volume

- Mathematical notation

$$\text{volume}(r, h) = r^2 * \pi * h$$

- Java declaration

```
double circleArea(double radius) {  
    return radius * radius * 3.1416;  
}
```

You can also use **Math.PI**

```
double cylinderVolume(double height, double radius) {  
    return height * circleArea(radius);  
}
```

- circleArea(2.0)** returns the value 12.5664
- cylinderVolume(1.5, 2.0)** evaluates the expression **1.5 * circleArea(2.0)** and returns the value 18.8496

Constructors

- Constructors are special methods that initialize attributes after reserving memory with **new**
- Constructors do not define a **return type** (you do not return anything, because they will automatically create a new object and return it)
- Formal parameters can be used for initialization
- **Example**

```
class Rational {  
    double numerator, denominator;  
  
    Rational(double n, double d) {  
        numerator = n;  
        denominator = d;  
    }  
}
```


The **void** return type

- The void return type is used when a method does not return a value
- The method is then called procedure

- **Example**

```
void alarm(String message) {  
    System.out.println("DANGER: " + message);  
}
```

- No **return** necessary
- Procedures can still change the state of an object (i.e. have side effects)

- Declaration

```
void methodName(formalParameterList) {  
    // body  
}
```

- Usage

```
methodName(actualParameterList);
```

Current object



- Normally the method depends on the state of the current object
→ This related object must be specified when calling
- This means that within a method it must be possible to address this reference object ("itself")
- For this purpose there is the keyword **this**

Example: using **this** in the constructor

```
class Point {  
    double x, y;  
    Point(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

Use of object state in methods

- **Example:** define method **shift** in class **Point**

```
class Point {  
    double x, y;  
  
    public Point(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    void shift(double dx, double dy) {  
        this.x = this.x + dx;  
        this.y = this.y + dy;  
    }  
}
```

- Call
- Call syntax

```
Point p = new Point(3.0, 4.0);  
p.shift(2.0, 4.0);
```

```
object.methodName(parameter);
```

If no object is specified → **this** is always used implicitly

Reference class - static methods

- A static method is independent of the state of concrete objects

- Example

```
class Weather {  
    static int fahrenheit(int celsius) {  
        return (celsius * 9) / 5 + 32;  
    }  
}
```

Question: can static methods have side effects?

- Call

```
Weather.fahrenheit(15);
```

- Syntax

```
static returnType methodName(parameter) {  
    // method body  
}
```

- Call syntax

```
ClassName.methodName(parameters)
```


The main method

- One static method is special: the **main()** method

```
class K {  
    public static void main(String[] args) {  
        //...  
    }  
}
```

Field / array

- **main(p1, ..., pn)** in class K will be called when you execute `java K` after compiling K
- The arguments **p1, ..., pn** are stored in a field called **args**

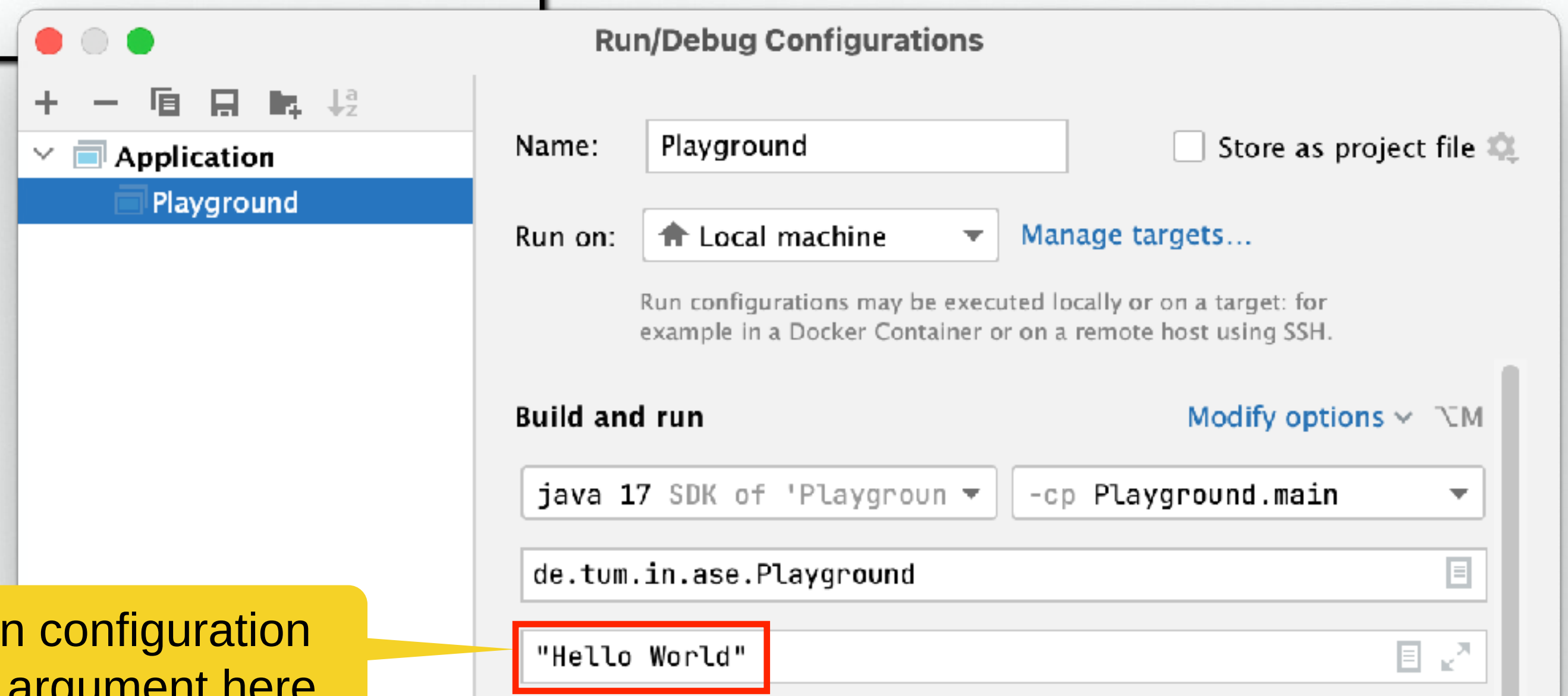
Covered later

Example

Playground.java

```
public class Playground {  
    public static void main(String[] args) {  
        if (args.length > 0) {  
            System.out.println("First argument: " + args[0]);  
        }  
        else {  
            System.out.println("No arguments provided");  
        }  
    }  
}
```

Run the application with an argument



Open the run configuration and add the argument here

Example: Point

Move points

```
class Point {  
    double x, y;  
  
    void shift(double dx, double dy) {  
        this.x = this.x + dx;  
        this.y = this.y + dy;  
    }  
}
```

Current coordinates

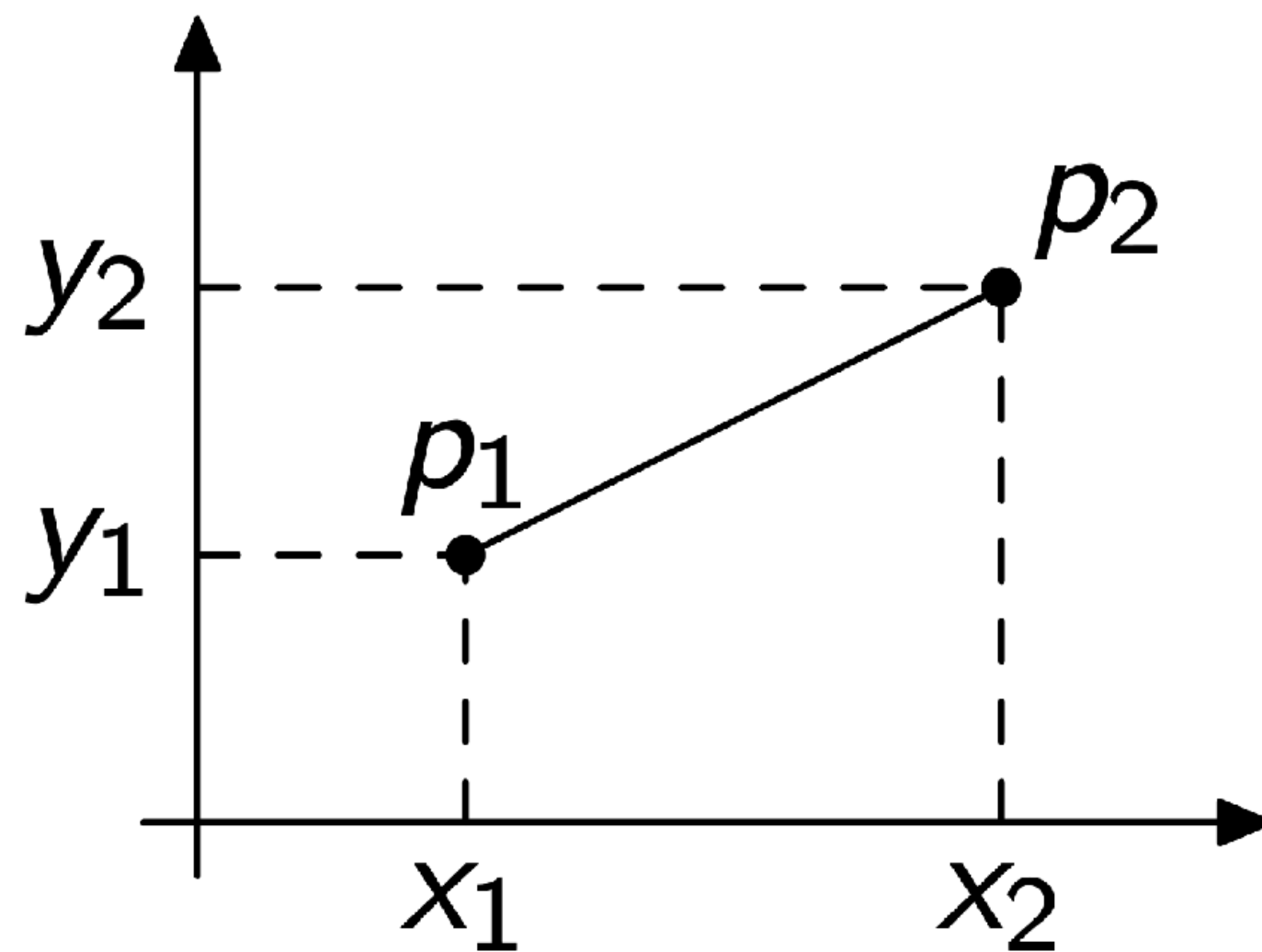
```
double getX() {  
    return this.x;  
}
```

```
double getY() {  
    return this.y;  
}
```


Move lines

Example: lines in $\mathbb{R}^2$

Concept



```
class Line {  
    Point p1;  
    Point p2;  
  
    Line(Point p1, Point p2) {  
        this.p1 = p1;  
        this.p2 = p2;  
    }  
  
    void shift(double dx, double dy) {  
        this.p1.shift(dx, dy);  
        this.p2.shift(dx, dy);  
    }  
}
```

References and method calls

- In Java, only **call-by-value** is used for parameter passing in method calls
- Definition of **call-by-value**: when calling methods, all statements in the parameters are first evaluated completely
 - Then, the values of those statements are passed as a **copy** to the called method
- **Note:** values of non-primitive types (i.e. objects) are references
 - A copy of the reference is passed
 - This can be used to still change the state of the referenced object

Local variables (1)

- Serve as auxiliary memory for intermediate results in methods
- Are stored on the so-called **runtime stack**
- Exist only as long as the method is processed
- **Example** (calculation of the area of a circle)

```
double circleArea(double radius) {  
    double radiusSquare = radius * radius;  
    return radiusSquare * 3.1416;  
}
```

Local variables (2)

- In Java, **parameters** are local variables that are initialized when the method is called (**call-by-value**)

- **Example**

```
int foo(int a) {  
    int x = a + 1;  
    a = x * x;           // bad programming style  
    return a + x;  
}
```

You should **not** change input parameters in a method

```
foo(a)  
  
int a      

|  |
|--|
|  |
|  |

 (Parameter)  
int x  
  
x = a + 1;  
a = x * x;  
return a + x;
```

Attention: simple argument types (**int**, **float**, ...) are handled differently than complex types (**Book**, **Author**, ...)

Local variables - differences to attributes

	Local variable	Attribute
Declaration	Within methods	Outside of methods
Lifetime	Method call	Lifetime of the associated object
Accessible	Only within a method	For all methods of the class
Purpose	Cache for values	State of the object